

LG 부트캠프 12기

프로젝트 결과보고서 쿵야

C반 3팀

엄제원 이경환 윤지원

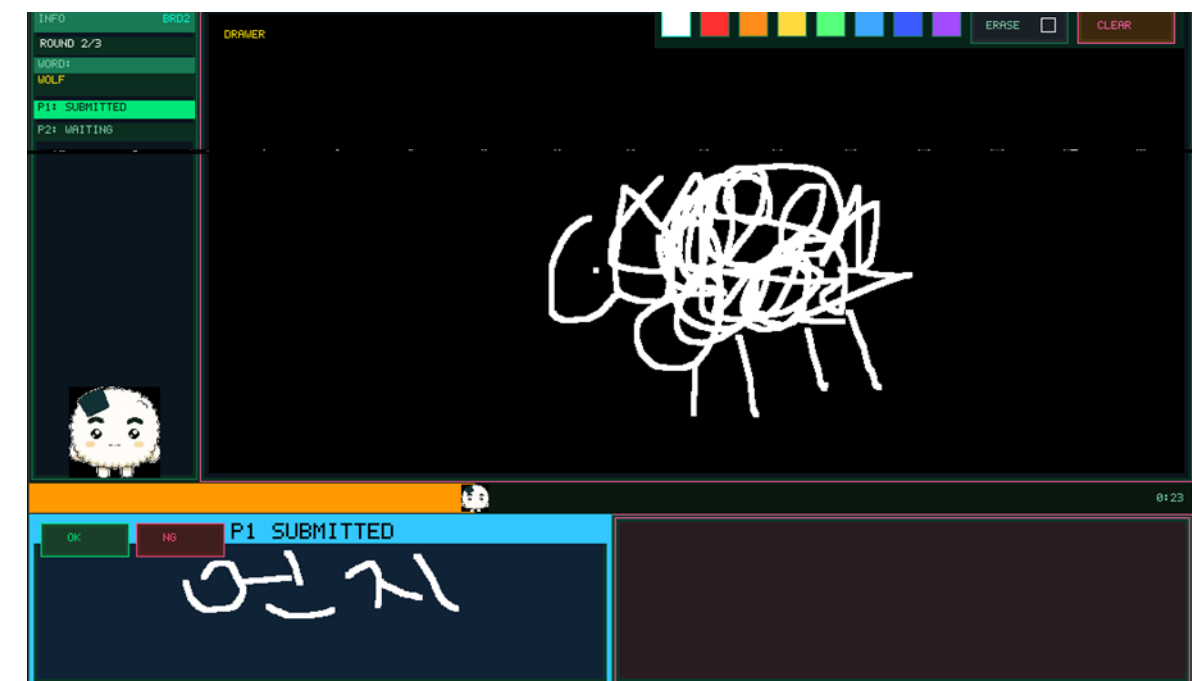
목차구성

CONTENTS COMPOSITION

1. 프로젝트 소개
2. 게임 규칙
3. 영상 시연
4. 핵심 기술
5. 결과 분석
6. 향후 연구 과제
7. 프로젝트 수행 후기

1. 프로젝트 소개

- 캐치 마인드
 - 필요 인원 3명 (1명 출제자, 2명 참가자)
- 3인 실시간 멀티플레이 지원
 - 허브 기반 UDP 통신을 통해 세명의 플레이어가 실시간으로 게임을 즐기도록 지원
- 플레이어 간 경쟁을 위한 랭킹 시스템
 - 문제 출제와 정답을 맞추며 점수 획득이 가능하며 5라운드 합산을 통해 랭킹 출력
- 터치펜을 활용한 정교한 그림 그리기
 - 색상 변경, 지우개, 초기화 기능을 통해 출제자가 편리하게 문제를 출제하도록 지원



2. 캐치마인드 게임 규칙



출제자 선정

첫 라운드는 선착순, 이후는 라운드 결과에 따라 출제자가 변경

문제 선택

출제자가 분야 → 주제어 순서로 선택
각 단계마다 4개 선택지가 랜덤 제시되며, 주제어는 참가자에게 비공개

제한 시간

각 라운드는 60초 제한
한 명이 정답을 맞히면 즉시 종료

- 출제자 1명과 참가자 2명이 겨루는 **5라운드 누적 점수제** 실시간 추측 게임
- 출제자가 그림을 그리면 참가자들은 선착순으로 정답을 제출

2. 점수 & 핵심 규칙

- 힌트 시스템
 - 30초 경과 시 참가자들에게 **카테고리(분야)**가 공개
- 정답 판단
 - 출제자가 직접 각 제출의 정답 여부를 판단
- 정답 시
 - 정답을 맞힌 참가자가 **다음 라운드 출제자**가 됨
- 시간 초과 시
 - 라운드가 실패로 종료되고 정답이 공개
 - 출제자는 1점 감점되며 다음 라운드에서도 출제자를 유지
- 5라운드 완료 시
 - 각 플레이어의 누적 점수가 환산되어 랭킹 출력

빠른 정답
(60~30초)



**출제자: 2점/
참가자: 3점 획득**

느린 정답
(30초 이후)



**출제자: 1점/
참가자: 2점 획득**

**시간 초과: 출제자 1점 감점,
정답 공개, 출제자 유지** ⌚!

3. 영상 시연

- 시연 영상

4. 핵심 기술

핵심 정리

Double Frame Buffer 기반 그래픽 구현

그림 그리기 도중 화면의 깜빡임 방지 및 부드러운 모션

UDP 기반 실시간 통신 & 동기화

UDP 브로드캐스트 + 동기화도 저지연 멀티플레이 구현

터치 입력 데이터 처리

보드 간 좌표 데이터 파싱 및 전송

GPIO 인터럽트 기반 물리 버튼 처리

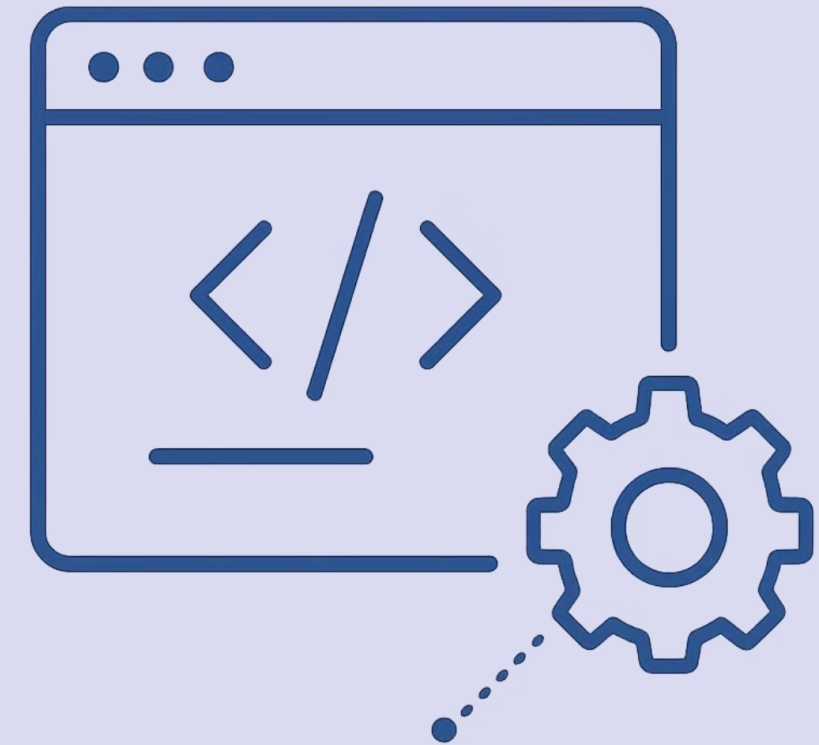
출제자/참가자에 따른 별도 버튼 기능 구현

최신 Linux 커널 포팅 & RFS 커스터마이징

최신 버전 리눅스 커널 포팅 및 커스터마이징

라이브러리 독립 구현

임베디드 디바이스 환경을 고려한 라이브러리 독립 구현



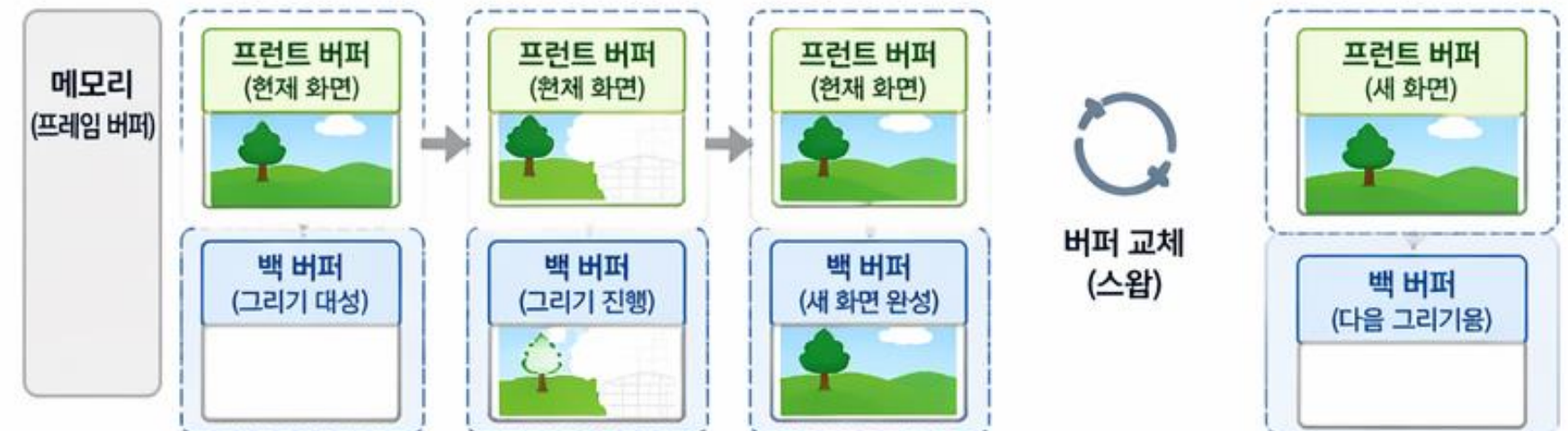
4. 핵심 기술 : Double Frame Buffer 기반 그래픽 구현

더블 프레임 버퍼를 사용하지 않을 때



- 기존 화면을 지우면서 흰 화면(또는 배경)이 먼저 보임
→ 깜빡임 발생
- 복잡한 장면이나 CPU 사용량이 높을 때 깜빡임이 더 심해짐

더블 프레임 버퍼를 사용할 때

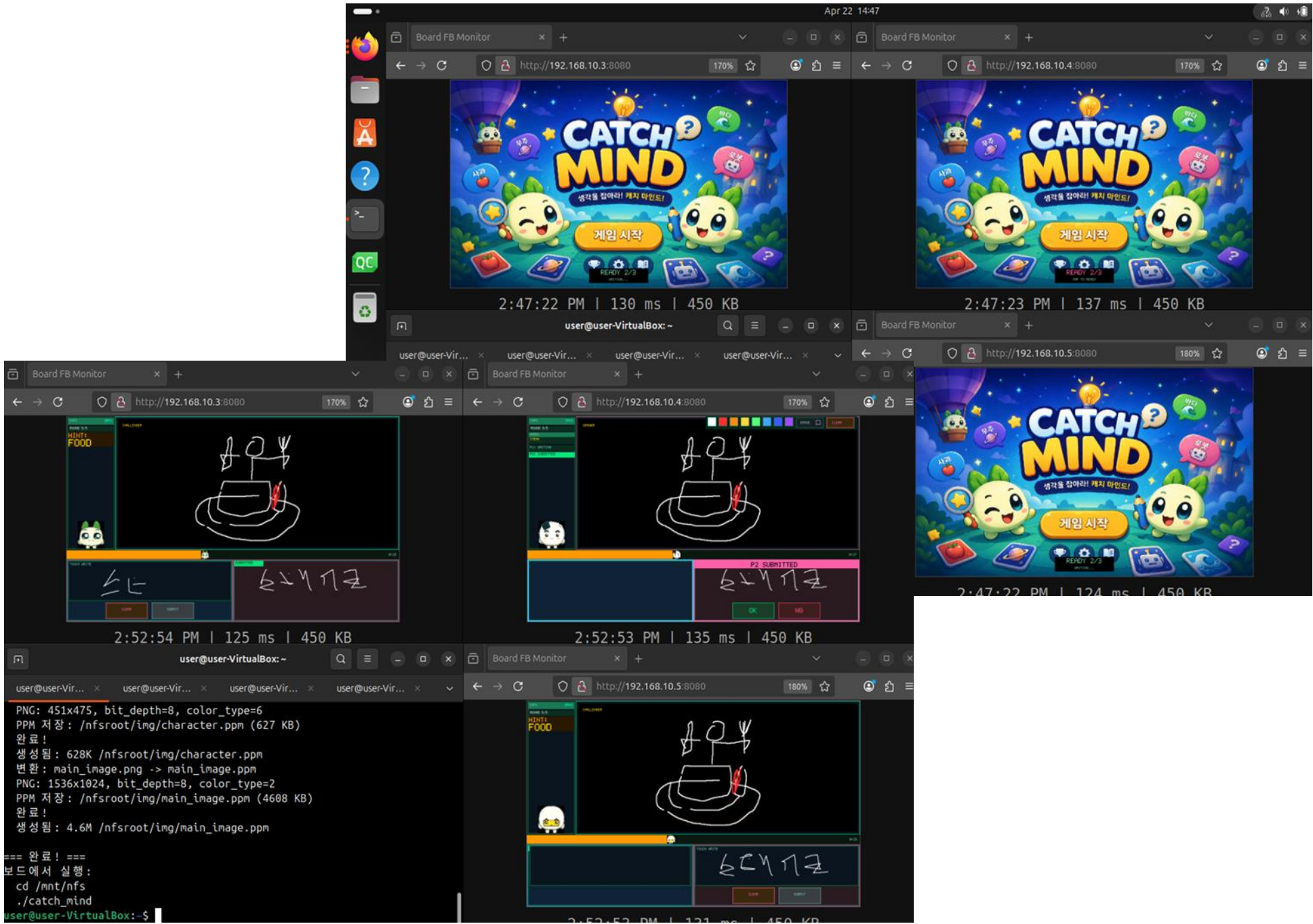


- 백 버퍼에서 모든 그리기를 완료한 후, 프런트/백 버퍼를 교체(스왑)하여 완성된 화면을 한 번에 출력
- 완성된 장면을 한 번에 출력하여 불필요한 중간 장면(흰 화면, 그리기 과정 노출)을 없애 부드럽게 화면 전환

4. 핵심 기술 : Frame Buffer 기반 모니터링

- 참가자 화면 실시간 모니터링
 - 게임 참가자의 보드를 ubuntu 화면으로 모니터링
 - 주변 사람들이 게임을 관전할 수 있도록 구현
- 개요
 - 순수 C (libc + pthreads), 외부 라이브러리 없음
 - TCP 포트 8080, accept() 마다 pthread_create() + pthread_detach()
- HTTP 엔드포인트

경로	응답	설명
GET /	HTML + JavaScript	1초 간격 자동 갱신 페이지
GET /frame	image/bmp	현재 프레임버퍼 캡처 이미지



4. 핵심 기술 : UDP 기반 실시간 통신 & 동기화

UDP 기반 실시간 통신 & 동기화

저지연 브로드캐스트

UDP 브로드캐스트로 멀티캐스트 통신 구현
TCP 오버헤드 없이 실시간 데이터 전송

논블로킹 I/O

select() 기반 멀티플렉싱으로 블로킹 없이 다중 소켓 처리

패킷 유실 대응

중요 메시지는 반복 전송으로 신뢰성 확보
순서 번호 기반 중복 제거

동기화 배리어

DRAWING_START 패킷으로 모든 클라이언트 렌더링 시점 동기화

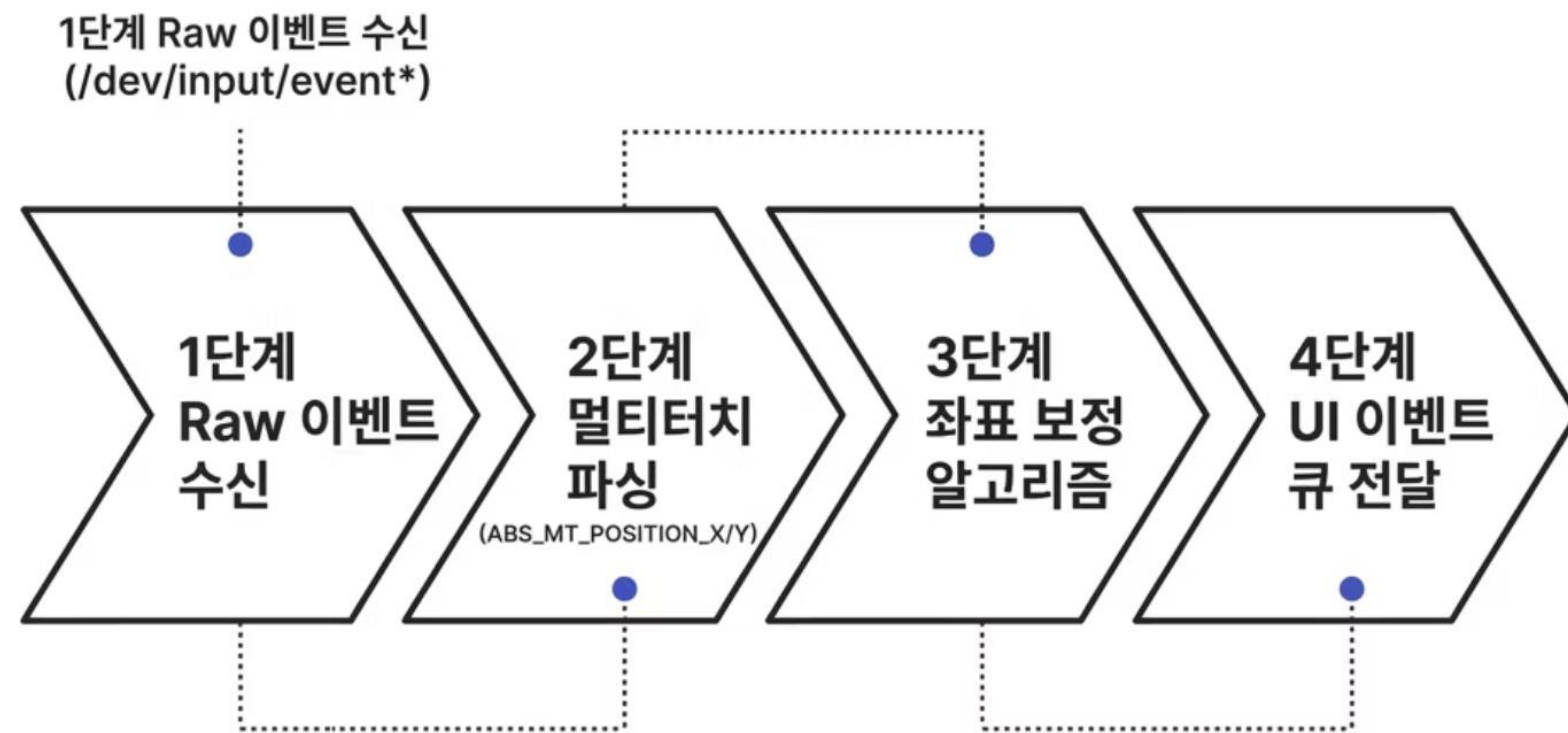
TCP 없이도 실시간 게임 동기화 구현

4. 핵심 기술 : UDP 기반 어플리케이션 프로토콜 설계

- 게임 구현에 필요한 프로토콜 설계
 - 3대의 보드가 실시간으로 상태를 공유하기 위해 UDP 브로드캐스트 기반의 프로토콜을 직접 설계
 - 메시지 형식은 CM|<nidId>|<kind>|<value> 구조로 정의하고 이를 기반으로 30여 종의 메시지 타입 구현
- 메시지의 중요도에 따라 별도의 설정값 적용
 - 메시지 타입에 따라 재전송 횟수와 간격을 차등 적용
 - 드로잉 좌표처럼 실시간성이 중요한 메시지는 누락을 허용
 - 판정/역할 배정처럼 상태 전이가 걸리는 메시지는 3~5회 재전송

종류	값	방향	재전송	설명
ROLE	DRAWER	출제자 → 전체	3회/50ms	출제자 선택 알림
STATUS	CHALLENGER_JOIN	도전자 → 전체	1회	도전자 참가 및 슬롯 협상
STATUS	DRAWING_START	출제자 → 전체	4회/150ms	라운드 시작 싱크 신호
STATUS	GAME_READY	출제자 → 전체	1회	출제자 라운드 화면 진입 완료
STATUS	DRAWING_ACTIVE	출제자 → 전체	1회	첫 스트로크 시작 알림
DRAW	nx,ny,color	출제자 → 전체	1회	드로잉 좌표 (0~999 정규화)
CLEAR	1	출제자 → 전체	3회/50ms	캔버스 초기화
STATUS	HINT#{카테고리}	출제자 → 전체	1회	30초 힌트
STATUS	JUDGING_ACTIVE	출제자 → 전체	3회/50ms	판정 시작, 도전자 submit 잠금
STATUS	JUDGING_END	출제자 → 전체	3회/50ms	판정 종료, 잠금 해제
STATUS	A_CLEAR_P{N}	출제자 → 전체	3회/30ms	P{N} 답변 패널 초기화
STATUS	RETRY_P{N}	출제자 → 전체	3회/30ms	P{N} 재도전 요청
STATUS	CORRECT_P{N}#BOARD{B}#EARLY/LATE	출제자 → 전체	1회	정답 판정 (보드번호, 타이밍 포함)
STATUS	WRONG_ALL#{정답}	출제자 → 전체	1회	시간 초과, 정답 공개
STATUS	ROUND_END	출제자 → 전체	1회	라운드 종료
STATUS	GAME_OVER	출제자 → 전체	1회	전체 라운드 종료
ANSWER	{pn}:DRAWN	도전자 → 전체	3회/30ms	답변 제출
A_POINT	{pn},nx,ny	도전자 → 전체	1회	답변 필기 좌표 (submit 시 일괄 전송)
A_CLEAR	{pn}	도전자 → 전체	1회	도전자 자신의 답변 지우기
A_UP	{pn}	도전자 → 전체	1회	필기 스트로크 끝
FINAL_SCORE_SUBMIT	{nidId}:{score}	각 보드 → 전체	400ms마다 반복	최종 점수 제출
FINAL_SCORE_SYNC	PLAYER1:n,PLAYER2:n,...	PLAYER1 → 전체	5회/20ms	전체 점수 스냅샷
FINAL_READY	1	각 보드 → 전체	800ms마다 반복	결과 화면 배리어

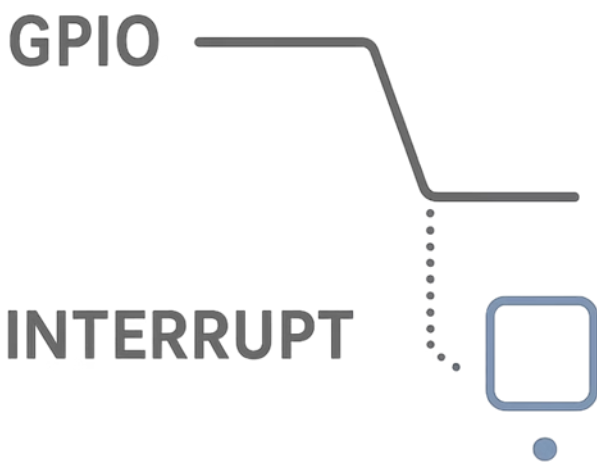
4. 핵심 기술 : Linux Input Subsystem 기반 터치 처리



• 이벤트 파싱 흐름

- /dev/input/event*에서 멀티터치 이벤트 직접 파싱
- 파싱 데이터 기반 좌표 추출
- Raw 값 → 화면 좌표 변환 (보정 알고리즘 적용)
- UI 이벤트 큐로 전달하여 처리
- 상대 보드에 출력

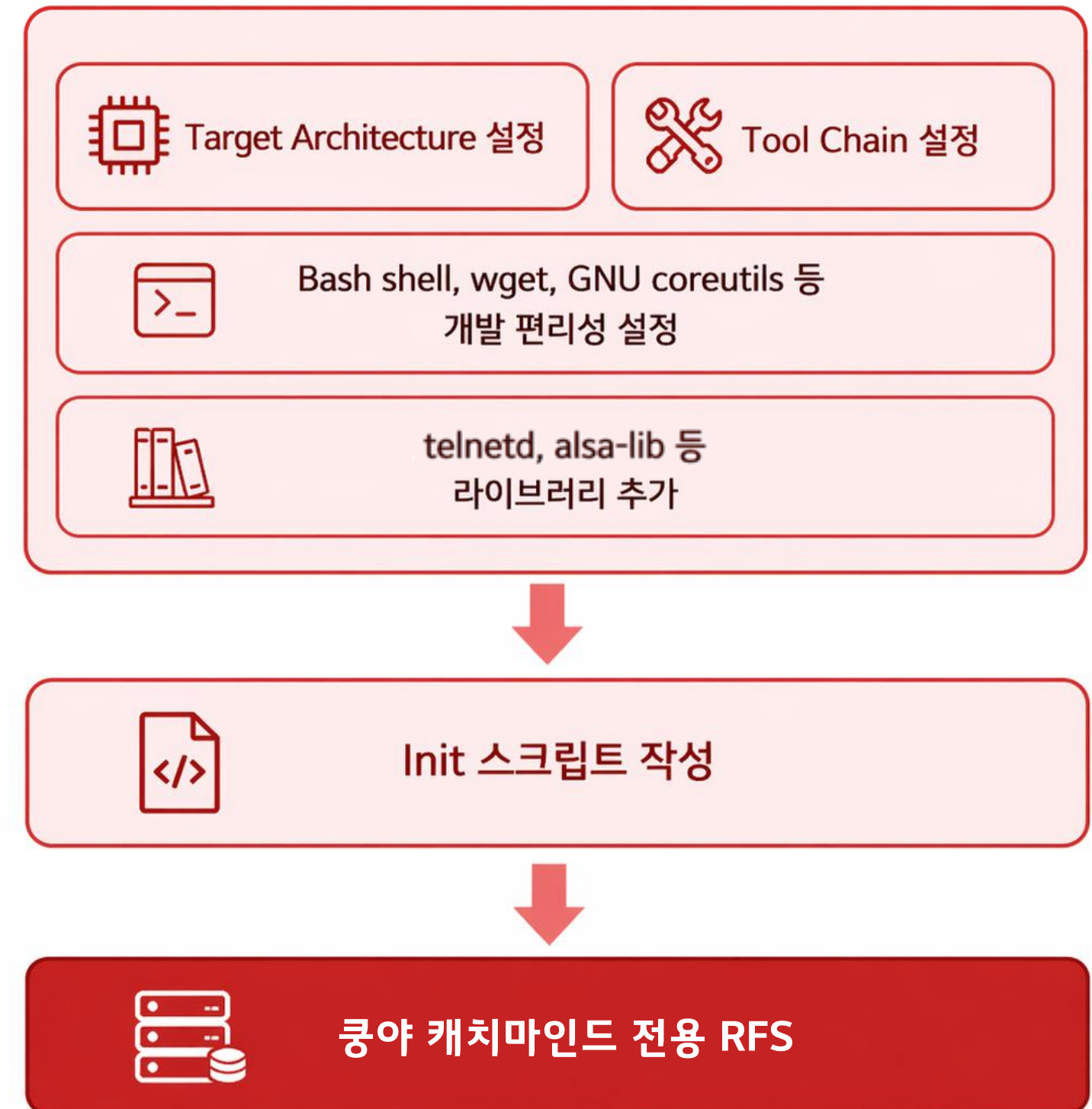
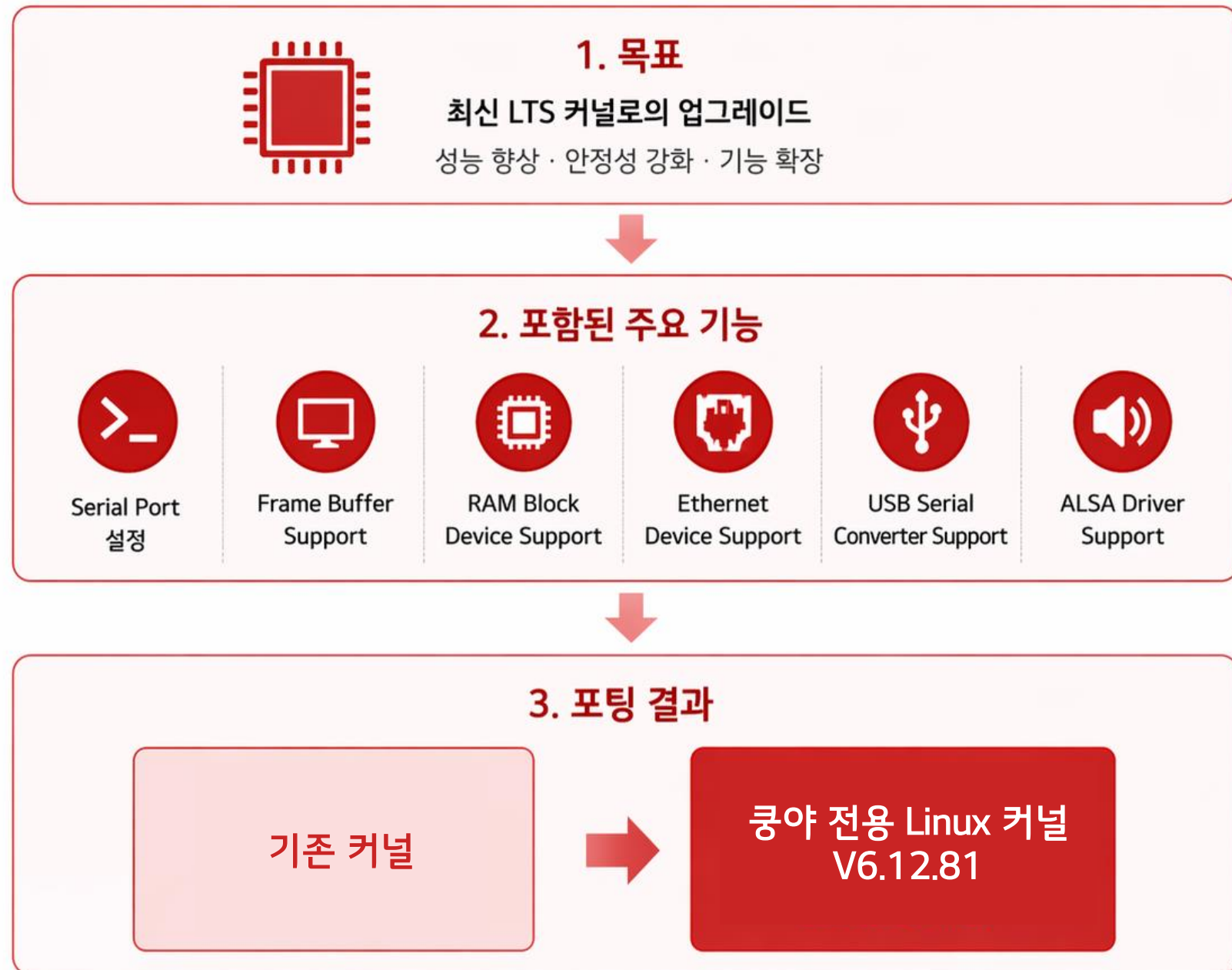
4. 핵심 기술 : GPIO 인터럽트 기반 물리 입력 처리



- 인터럽트 처리 구조
 - /dev/gpiochip0 + GPIO_GET_LINEEVENT_IOCTL로 커널 이벤트 등록
 - 하강 에지(Falling Edge) 인터럽트 기반 버튼 이벤트 감지
 - poll()로 이벤트 대기, 별도 스레드에서 처리
 - std::atomic으로 스레드 간 안전한 상태 공유

버튼	GPIO line	출제자	도전자
SW2	17	OK (정답)	CLEAR (지우기)
SW3	18	NG (오답)	SUBMIT (제출)

4. 핵심 기술 : 최신 Linux 커널 포팅 & RFS 커스터마이징



4. 핵심 기술 : 라이브러리 독립 구현

• 비트맵 폰트 직접 제작

- A-Z, 0-9, 기호(. : - / ! 등)를 5×7 비트맵 패턴으로 display.cpp에 하드코딩
- scale 파라미터로 정수 배율 확대 지원 (UI에서 1×~4× 혼용)
- 모든 UI 레이블은 ASCII로 통일해 폰트 복잡도 최소화

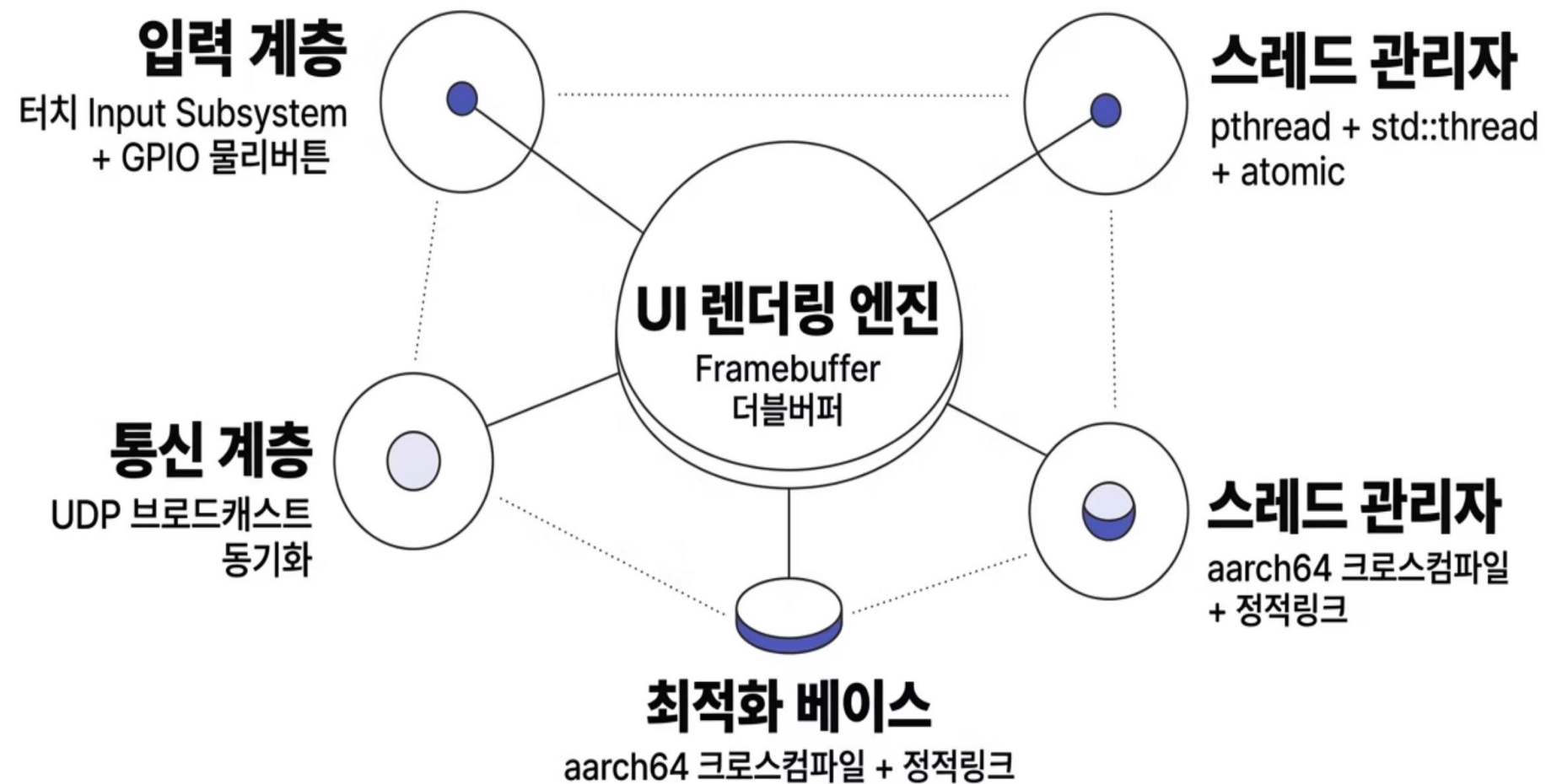


• PPM 이미지 포맷 직접 파싱

- 보드에 libpng가 없어 PNG를 직접 디코딩할 수 없음
- 원본 이미지는 PNG로 관리하고, 배포 시 deploy.sh가 자동으로 PPM으로 변환 후 NFS에 배포
- 로딩 시 보드 bpp(16/32)에 맞게 RGB → RGB565 또는 ARGB8888로 변환 후 프레임버퍼에 직접 기록



4. 핵심 기술 : 시스템 통합 아키텍처



• 설계 원칙

- 모든 계층이 외부 라이브러리 없이 직접 구현
- 각 계층이 독립 스레드로 동작하며 이벤트 큐로 연결
- 저사양 환경에서 메모리·CPU 오버헤드 최소화

• 주요 기술 스택

- /dev/fb0, mmap, select, poll
- /dev/input/event*, /dev/gpiochip0
- pthread, std::thread, std::atomic
- UDP 브로드캐스트 + 동기화

5. 결과 분석

• 네트워크 성능

항목	측정/설계 값	결과	평가
평균 지연 시간	약 50 ~ 100ms	실시간 드로잉 자연스러움	✓ 실시간 게임 기준 적합
패킷 전달 신뢰성	재전송 3~5회	상태 메시지 거의 100% 전달	✓ 동기화 안정적
드로잉 패킷	단발 전송 (손실 허용)	일부 점 손실 발생	✓ UX 영향 미미

• 그래픽 성능

항목	방식	결과	평가
화면 출력	더블 버퍼링 (HW/SW)	깜빡임 없음	✓ 완전 개선
렌더링 방식	mmap + 직접 드로잉	중간 프레임 노출 없음	✓ 안정적
화면 전환	버퍼 교체	부드러운 UI	✓ UX 향상

• 최종 목표 달성 평가

목표	달성 여부	비고
실시간 멀티플레이	✓	100ms 이하 지연
안정적 동기화	✓	상태/점수 완전 일치
깜빡임 없는 UI	✓	더블 버퍼링 적용
충돌 없는 판정	✓	동시 입력 문제 해결
확장성	△	UDP 기반 구조 한계

6. 향후 연구 과제

하드웨어 개선

더 좋은 디스플레이로 교체하여 터치 정확도 개선

현재
(SBC + 기본 디스플레이)



- 해상도 낮음
- 터치 정확도 제한적
- 시인성/내구성 부족

개선
(고성능 디스플레이)



- 고해상도 디스플레이
- 정밀하고 빠른 터치
- 밝기/시인성/내구성 향상

사용자 확장

3인 GAME → 다수 인원 동시 참여 지원

현재
(3인 고정)



3인 제한
(출제자 1명 + 참가자 2명)

개선
(다중 인원 지원)



N명 동시 참여
(대규모 멀티플레이 지원)



최종 목표

확장 가능한 온라인
멀티플레이 플랫폼 구축



온라인 서비스
전 세계 플레이어 연결

+



향상된 사용자 경험
정확한 터치 & 고화질

+



대규모 멀티플레이
더 많은 인원, 더 많은 재미

+



지속 가능한 서비스
확장성 & 수익 모델 기반

=



완성도 높은
게임 플랫폼 실현

7. 프로젝트 수행 후기

세부 규칙 확립을 위한 시행 착오

- 참가자 및 출제자가 그린 그림이 공유가 되는 규칙을 정하고 구현하고 테스트 하는 과정에서 시행착오가 많았음
- 게임을 플레이하며 규칙을 조금씩 업데이트 해나가는 것으로 해결함

버그 해결

- 순식간에 지나간 버그를 잡기 위해 반복 게임 플레이를 계속 진행함
- 반복 플레이를 통해 버그를 특정하고 해결함

부드러운 터치감을 위한 최적화

- 데이터 전송 주기 조절을 통해 게임 플레이에 집중 가능하도록 최적화 진행

점수 불일치 오류 해결

- 네트워크 지연이나 패킷 누락으로 라운드가 끝나는 시점에 보드 간 점수가 서로 달라지는 상황이 발생
- 각 보드는 자기 자신의 점수만 직접 관리하는 방식으로 해결

교육 기간에 배운 내용은 프로젝트에 적용함으로써 복습이 되고 피와 살이 되고 기억에 잘 남음

감사합니다!

Q&A